



Universidad Autónoma de Tamaulipas

Producto integrador: punto de venta

Codigo c#

Facultad de Ingeniería Tampico

Ingeniería en Sistemas Computacionales

Asignatura: Fundamentos De programación

Grupo: N Grado: 1

Nombre del Docente: Álvarez Navarro Eduardo

Alumno: Guevara Martinez Angel Jeremy

Matricula: 2243330342

using System;

```
public class Proyecto_Integrador
{
    static string[][] productos = new string[20][];
    static string[][] ventas;
    static int tamventas = 100;
    static string fecha;

    public static string MostrarMenu(string[] opciones)
    {
        string cadena = "";
        foreach (string info in opciones)
        {
            cadena += info + "\n";
        }
        return cadena;
    }

    public static bool EsNumeroEntero(string dato)
    {
        if (string.IsNullOrEmpty(dato))
            return false;

        foreach (char c in dato)
        {
```

```
        if (!char.IsDigit(c))
            return false;
    }
    return true;
}

public static bool EsNumeroDouble(string dato)
{
    bool punto = false;

    if (string.IsNullOrEmpty(dato))
        return false;

    foreach (char c in dato)
    {
        if (char.IsDigit(c))
        {
            continue;
        }
        else if (c == '.')
        {
            if (!punto)
            {
                punto = true;
            }
        }
        else
```

```
{  
    return false;  
}  
}  
else  
{  
    return false;  
}  
}  
  
return true;  
}  
  
public static bool EvaluarNumerico(string dato, int tipo)  
{  
    switch (tipo)  
    {  
        case 1:  
            return EsNumeroEntero(dato);  
  
        case 2:  
            return EsNumeroDouble(dato);  
  
        default:  
            return false;  
    }  
}
```

}

```
public static string Dialogo(string texto)
```

```
{
```

```
    Console.Write(texto + " : ");
```

```
    return Console.ReadLine();
```

```
}
```

```
public static string Leer(string texto)
```

```
{
```

```
    string cadena = Dialogo(texto);
```

```
    if (cadena != null)
```

```
    {
```

```
        cadena = cadena.Trim();
```

```
        if (cadena == "")
```

```
            cadena = null;
```

```
    }
```

```
    else
```

```
    {
```

```
        cadena = null;
```

```
    }
```

```
    return cadena;
```

```
}
```

```
public static string LeerValidado(string texto_dialogo, int tipo_dato)
```

```
{  
    string cadena;  
  
    do  
    {  
        cadena = Leer(texto_dialogo);  
  
        if (cadena == null)  
        {  
            Console.WriteLine("dato nulo");  
        }  
        else if (!EvaluarNumerico(cadena, tipo_dato))  
        {  
            Console.WriteLine("dato incorrecto");  
            cadena = null;  
        }  
  
    } while (cadena == null);  
  
    return cadena;  
}
```

```
public static string DesplegarMenu(string titulo1, string[] menu)  
{  
    string cadena = titulo1 + "\n\n";
```

```
cadena += MostrarMenu(menu);

cadena += "\n Que opcion deseas ";

return Dialogo(cadena);
}

public static string RellenarEspacios(string dato, int tamano)
{
    if (dato == null)
        dato = "";
    return dato.PadRight(tamano);
}

public static string Fecha()
{
    DateTime fecha = DateTime.Now;
    return fecha.ToString("dd-MM-yyyy");
}

public static string IdTicketSiguiente(string idticket)
{
    string idticketnext;
    int num = int.Parse(idticket) + 1;

    if (num < 10)
        idticketnext = "00" + num.ToString().Trim();
    else if (num > 9 && num < 100)
```

```
idticketnext = "0" + num.ToString().Trim();
```

```
else
```

```
idticketnext = num.ToString().Trim();
```

```
return idticketnext;
```

```
}
```

```
public static int ObtenerUltimaPosicion(string[][] matriz)
```

```
{
```

```
int ultimaPosicion = -1;
```

```
for (int i = 0; i < matriz.Length; i++)
```

```
{
```

```
if (matriz[i] != null && matriz[i][0] != null && matriz[i][0] != "")
```

```
{
```

```
ultimaPosicion = i;
```

```
}
```

```
}
```

```
return ultimaPosicion;
```

```
}
```

```
public static string[][] CargarProductos()
```

```
{
```

```
string[][] producto =
```

```
{
```



```
new string[] {"001", "Frappe Chocolate", "65", "20", "16"},
new string[] {"002", "Frappe Vainilla", "65", "20", "16"},
new string[] {"003", "Capuchino Combo", "119", "15", "16"},
new string[] {"004", "Latte Combo", "119", "15", "16"},
new string[] {"005", "Lechero Combo", "119", "15", "16"},
new string[] {"006", "Galletas 2x", "80", "25", "16"},
new string[] {"007", "Rebanada de Pastel", "59", "18", "16"},
new string[] {"008", "Tes y Tisanas 2x", "99", "20", "16"},
new string[] {"009", "Panini Tradicional", "125", "10", "16"},
new string[] {"010", "Bisquet con Mantequilla", "89", "15", "16"},
new string[] {"011", "Ramen Coreano Picante", "85", "12", "16"},
new string[] {"012", "Ramen Coreano Queso", "85", "12", "16"},
new string[] {"013", "Ramen Coreano Pollo", "85", "12", "16"},
new string[] {"014", "Ramen Coreano Carne", "85", "12", "16"},
new string[] {"015", "Bebida Fria", "35", "20", "16"},
new string[] {"016", "Sandwich Jamon", "95", "14", "16"},
new string[] {"017", "Croissant", "55", "15", "16"},
new string[] {"018", "Brownie", "45", "15", "16"},
new string[] {"019", "Chocolate Caliente", "58", "18", "16"},
new string[] {"020", "Cafe Americano", "45", "20", "16"}

};

return producto;
}

public static string MostrarProducto(string[] vproducto)
```

```
{  
    string codigo = RellenarEspacios(vproducto[0], 5);  
    string nombre = RellenarEspacios(vproducto[1], 30);  
    string precio = RellenarEspacios(vproducto[2], 10);  
    string cantidad = RellenarEspacios(vproducto[3], 10);  
    string iva = RellenarEspacios(vproducto[4], 5);  
  
    return codigo + nombre + precio + cantidad + iva;  
}  
  
public static string MostrarLista(string[][] vproductos)  
{  
    string salida = "";  
  
    for (int ciclo = 0; ciclo < vproductos.Length; ciclo++)  
    {  
        if (vproductos[ciclo] != null && vproductos[ciclo][0] != null)  
        {  
            string[] vproducto =  
            {  
                vproductos[ciclo][0],  
                vproductos[ciclo][1],  
                vproductos[ciclo][2],  
                vproductos[ciclo][3],  
                vproductos[ciclo][4]  
            };  
        }  
    }  
};
```

```
        salida += MostrarProducto(vproducto) + "\n";
    }
}

return salida;
}

public static int ExisteProducto(string codigo, string[][] vproductos)
{
    if (string.IsNullOrEmpty(codigo))
        return -1;

    int enc = -1;
    int pos = 0;
    int tam = vproductos.Length;

    for (int ciclo = 0; ciclo < tam; ciclo++)
    {
        if (vproductos[ciclo] != null && vproductos[ciclo][0] != null &&
            vproductos[ciclo][0].CompareTo(codigo.Trim()) == 0)
        {
            enc = pos;
        }
        pos++;
    }
}
```

```
return enc;
}

public static double ObtenerIva(string codigo, string[][] productos)
{
    int posicion = ExisteProducto(codigo, productos);

    if (posicion > -1)
    {
        return double.Parse(productos[posicion][4]);
    }

    return 0;
}

public static int DescontarStock(string[][] vproductos, string codigo, int cantidad)
{
    int posicion = ExisteProducto(codigo, vproductos);

    if (posicion == -1)
        return -2;

    int stock = int.Parse(vproductos[posicion][3]);

    if (stock == 0)
```

```
return 0;
```

```
if (cantidad > stock)
```

```
    return -1;
```

```
stock -= cantidad;
```

```
vproductos[posicion][3] = stock.ToString();
```

```
return 1;
```

```
}
```

```
public static void ModificarProducto(string[][] vproductos)
```

```
{
```

```
    string codigo, precio;
```

```
    int posicion;
```

```
    string info = MostrarLista(vproductos);
```

```
    codigo = Leer(info + "\nIntroduce el codigo del producto a modificar");
```

```
    if (codigo != null)
```

```
    {
```

```
        posicion = ExisteProducto(codigo, vproductos);
```

```
        if (posicion > -1)
```

```
        {
```

```
            string[] vproducto =
```

```
{  
    vproductos[posicion][0],  
    vproductos[posicion][1],  
    vproductos[posicion][2],  
    vproductos[posicion][3],  
    vproductos[posicion][4]  
};  
  
precio = LeerValidado("\nIntroduce el precio de " + MostrarProducto(vproducto) + " ", 2);  
vproductos[posicion][2] = precio;  
  
    Console.WriteLine("precio actualizado");  
}  
else  
{  
    Console.WriteLine("no existe el codigo");  
}  
}  
else  
{  
    Console.WriteLine("dato nulo");  
}  
}  
  
public static void MenuProductos(string[][] vproductos)  
{
```

```
string[] datosmenuproductos = { "1.-Modificar ", "2.-Listado ", "3.-Salida " };  
  
string opcion = "0";  
  
do  
{  
    opcion = DesplegarMenu("Opciones de Productos", datosmenuproductos);  
  
    if (opcion == null)  
    {  
        Console.WriteLine("opcion incorrecta ");  
    }  
    else  
    {  
        switch (opcion)  
        {  
            case "1":  
                ModificarProducto(vproductos);  
                break;  
  
            case "2":  
                Console.WriteLine(MostrarLista(vproductos));  
                break;  
  
            case "3":  
                Console.WriteLine("Salida del Sistema ");  
                break;
```

default:

```
Console.WriteLine("No existe esta opcion ");
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
while (opcion.CompareTo("3") != 0);
```

```
}
```

```
public static string[][] CrearVenta()
```

```
{
```

```
string[][] ventas = new string[tamventas][];
```

```
for (int i = 0; i < tamventas; i++)
```

```
{
```

```
ventas[i] = new string[5];
```

```
}
```

```
return ventas;
```

```
}
```

```
public static string UltimoTicket(int pos, string[][] mventa)
```

```
{
```

```
string idticket = "000";
```

```
if (pos > -1)
```



```
{  
    idticket = mventa[pos][0];  
}  
return idticket;  
}  
  
public static string[][] CrearTicket()  
{  
    string[][] ticket = new string[20][];  
  
    for (int i = 0; i < 20; i++)  
    {  
        ticket[i] = new string[4];  
    }  
  
    return ticket;  
}  
  
public static int ExisteTicketCodigo(string[][] mticket, string codigo)  
{  
    if (string.IsNullOrEmpty(codigo))  
        return -1;  
  
    int enc = -1;  
    int pos = ObtenerUltimaPosicion(mticket);
```

```
Console.WriteLine(" buscando " + codigo + " tamaño arreglo " + pos);
```

```
for (int ciclo = 0; ciclo <= pos; ciclo++)  
{  
    if (mticket[ciclo] != null && mticket[ciclo][0] != null &&  
        mticket[ciclo][0].CompareTo(codigo.Trim()) == 0)  
    {  
        enc = ciclo;  
        return enc;  
    }  
}  
  
return enc;  
}
```

```
public static bool InsertarProductoTicket(string[][] mticket, string[] datos, int tamticket)  
{  
    bool sucedio = true;  
    int posticket = ObtenerUltimaPosicion(mticket);  
    int enc = ExisteTicketCodigo(mticket, datos[0]);  
  
    if (posticket < tamticket - 1)  
    {  
        if (enc > -1)  
        {  
            int cantidadactual = int.Parse(mticket[enc][3]);
```

```
mticket[enc][3] = (cantidadactual + 1).ToString();
}
else
{
    postticket++;
    mticket[postticket][0] = datos[0];
    mticket[postticket][1] = datos[1];
    mticket[postticket][2] = datos[2];
    mticket[postticket][3] = datos[3];
    Console.WriteLine("Insertando: No existe el producto en el ticket");
}
}
else
{
    sucedio = false;
}

return sucedio;
}

public static string TotalProducto(string precio, string cantidad)
{
    double total = double.Parse(precio) * double.Parse(cantidad);
    return total.ToString("F2");
}
```

```
public static string MostrarProductoTicket(string[][] mticket, int pos)
{
    string codigo = RellenarEspacios(mticket[pos][0], 5);
    string producto = RellenarEspacios(mticket[pos][1], 30);
    string precio = RellenarEspacios(mticket[pos][2], 10);
    string cantidad = RellenarEspacios(mticket[pos][3], 5);
    string totalproducto = RellenarEspacios(TotalProducto(mticket[pos][2], mticket[pos][3]), 10);

    return codigo + producto + precio + cantidad + totalproducto;
}

public static string MostrarTicket(string[][] mticket)
{
    string salida = "";
    int pos = ObtenerUltimaPosicion(mticket);

    for (int ciclo = 0; ciclo <= pos; ciclo++)
    {
        salida += MostrarProductoTicket(mticket, ciclo) + "\n";
    }

    return salida;
}

public static double SubTotalTicket(string[][] mticket)
{

```

```
double subtotal = 0;

int pos = ObtenerUltimaPosicion(mticket);

for (int ciclo = 0; ciclo <= pos; ciclo++)
{
    subtotal += double.Parse(TotalProducto(mticket[ciclo][2], mticket[ciclo][3]));
}

return subtotal;
}

public static double IvaTicket(string[][] mticket, string[][] productos)
{
    double ivatotal = 0;
    int pos = ObtenerUltimaPosicion(mticket);

    for (int ciclo = 0; ciclo <= pos; ciclo++)
    {
        string codigo = mticket[ciclo][0];
        double iva = ObtenerIva(codigo, productos);

        if (iva != 0)
        {
            double precio = double.Parse(mticket[ciclo][2]);
            double cantidad = double.Parse(mticket[ciclo][3]);
            double totalproducto = precio * cantidad;
```

```
        ivatotal += ((iva / 100) * totalproducto);
    }
}

return ivatotal;
}

public static double TotalTicket(string[][] mticket, string[][] productos)
{
    double total = SubTotalTicket(mticket);

    if (total > 0)
    {
        total = IvaTicket(mticket, productos) + total;
    }

    return total;
}

public static string MostrarTicketVenta(string[][] mticket, string idticket, string fecha, string[][] productos)
{
    string salida = "";
    string subtotal = SubTotalTicket(mticket).ToString("F2");
    string iva = IvaTicket(mticket, productos).ToString("F2");
    string total = TotalTicket(mticket, productos).ToString("F2");
```

```
salida = "Fecha " + fecha + " Ticket No." + idticket;
```

```
salida += "\n" + MostrarTicket(mticket);
```

```
salida += "\n \n El total sin iva " + subtotal;
```

```
salida += "\n el iva total es " + iva;
```

```
salida += "\n el total de la venta fue " + total;
```

```
return salida;
```

```
}
```

```
public static string MostrarListaProductosVenta(string[][] vproductos)
```

```
{
```

```
    string salida = "";
```

```
    for (int ciclo = 0; ciclo < vproductos.Length; ciclo++)
```

```
    {
```

```
        if (vproductos[ciclo] != null && vproductos[ciclo][0] != null)
```

```
        {
```

```
            int existencia = int.Parse(vproductos[ciclo][3]);
```

```
            if (existencia > 0)
```

```
            {
```

```
                string[] vproducto = (string[])vproductos[ciclo].Clone();
```

```
                string cadena = MostrarProducto(vproducto);
```

```
                salida += cadena + "\n";
```

```
            }
```

```
}  
  
}  
  
return salida;  
}  
  
public static void CapturaVentaProducto(string[][] mticket, string[][] mproductos, string idticket, int tamticket)  
{  
    string codigo, info;  
    info = MostrarListaProductosVenta(mproductos);  
    codigo = Leer(info + "\nIntroduce el codigo del producto");  
  
    if (codigo != null)  
    {  
        int posp = ExisteProducto(codigo.Trim(), mproductos);  
  
        if (posp > -1)  
        {  
            string[] producto = (string[])mproductos[posp].Clone();  
            int resultado = DescontarStock(mproductos, codigo, 1);  
  
            if (resultado == 1)  
            {  
                Console.WriteLine(MostrarProducto(producto));  
  
                string[] venta = new string[4];
```



```
venta[0] = mproductos[posp][0];  
venta[1] = mproductos[posp][1];  
venta[2] = mproductos[posp][2];  
venta[3] = "1";  
  
if (!InsertarProductoTicket(mticket, venta, tamticket))  
{  
    Console.WriteLine("el Arreglo esta lleno");  
}  
}  
else if (resultado == 0)  
{  
    Console.WriteLine("no hay productos para venta");  
}  
else if (resultado == -1)  
{  
    Console.WriteLine("la cantidad es mayor al stock");  
}  
else if (resultado == -2)  
{  
    Console.WriteLine("el codigo no existe no se puede agregar");  
}  
}  
else  
{  
    Console.WriteLine("el codigo no existe no se puede agregar");
```

```
}  
  
}  
  
else  
  
{  
    Console.WriteLine("dato nulo");  
}  
}  
  
public static void RemoverProductoTicket(string[][] mticket, int pos)  
{  
    int tam = ObtenerUltimaPosicion(mticket);  
  
    if (tam > pos)  
    {  
        for (int i = pos; i < tam; i++)  
        {  
            mticket[i][0] = mticket[i + 1][0];  
            mticket[i][1] = mticket[i + 1][1];  
            mticket[i][2] = mticket[i + 1][2];  
            mticket[i][3] = mticket[i + 1][3];  
        }  
    }  
  
    mticket[tam][0] = null;  
    mticket[tam][1] = null;  
    mticket[tam][2] = null;
```

```
mticket[tam][3] = null;
}

public static void EliminarProductoTicket(string[][] mticket, int pos)
{
    int cantidad = int.Parse(mticket[pos][3]);

    if (cantidad > 1)
    {
        mticket[pos][3] = (cantidad - 1).ToString();
    }
    else
    {
        RemoverProductoTicket(mticket, pos);
    }
}

public static void Eliminar(string[][] mticket, string[][] mproductos)
{
    string codigo, info;
    info = MostrarTicket(mticket);
    codigo = Leer(info + "\nIntroduce el codigo del producto");

    if (codigo != null)
    {
        int pos = ExisteTicketCodigo(mticket, codigo);
```

```
if (pos > -1)
{
    int posproducto = ExisteProducto(codigo, mproductos);
    if (posproducto > -1)
    {
        string nuevacantidad = (int.Parse(mproductos[posproducto][3]) + 1).ToString();
        mproductos[posproducto][3] = nuevacantidad;

        EliminarProductoTicket(mticket, pos);
    }
    else
    {
        Console.WriteLine("no existe el producto en el inventario");
    }
}
else
{
    Console.WriteLine("no existe el producto en el ticket");
}
}
else
{
    Console.WriteLine("dato nulo");
}
}
```

```
public static void AgregarProductoAVenta(string[][] mticket, string[][] mventa, string idticket)
```

```
{  
    int posventas = ObtenerUltimaPosicion(mventa);  
    int posticket = ObtenerUltimaPosicion(mticket);
```

```
    for (int i = 0; i <= posticket; i++)  
    {  
        if (mticket[i] != null && mticket[i][0] != null)  
        {  
            posventas++;  
            mventa[posventas][0] = idticket;  
            mventa[posventas][1] = mticket[i][0];  
            mventa[posventas][2] = mticket[i][1];  
            mventa[posventas][3] = mticket[i][2];  
            mventa[posventas][4] = mticket[i][3];  
        }  
    }  
}
```

```
public static void Pagar(string idticket, string[][] mventa, string[][] mticket)
```

```
{  
    int posventas = ObtenerUltimaPosicion(mventa);  
    int post = ObtenerUltimaPosicion(mticket);  
  
    if ((posventas + post) < 100)
```

```
{
    AgregarProductoAVenta(mticket, mventa, idticket);
}
else
{
    Console.WriteLine("Desbordamiento de Memoria de ventas");
}
}

public static void DevolucionTicket(string[][] mticket, string[][] mproductos)
{
    int posmticket = ObtenerUltimaPosicion(mticket);

    for (int pos = 0; pos <= posmticket; pos++)
    {
        string codigo = mticket[pos][0];
        int posp = ExisteProducto(codigo.Trim(), mproductos);

        if (posp > -1)
        {
            int cant = int.Parse(mticket[pos][3]) + int.Parse(mproductos[posp][3]);
            mproductos[posp][3] = cant.ToString();
        }
    }
}
```

```
public static void CancelarVenta(string[][] mticket, string[][] mproductos)
{
    DevolucionTicket(mticket, mproductos);

    for (int i = 0; i < mticket.Length; i++)
    {
        for (int j = 0; j < mticket[i].Length; j++)
        {
            mticket[i][j] = null;
        }
    }
}

public static void MenuPuntoVenta(string[][] ventas, string idticket, string[][] productos)
{
    string opcion, membrete;
    bool pago = false;
    int tamticket = 50;

    string[][] Vticket = new string[tamticket][];
    for (int i = 0; i < tamticket; i++)
    {
        Vticket[i] = new string[4];
    }

    idticket = IdTicketSiguiente(idticket);
```

```
string fechadia = Fecha();

opcion = "";

do
{
    membrete = "Fecha del Dia " + fechadia + " Ticket No " + idticket;
    membrete = membrete + "\n-----\n";

    string ticketTexto = MostrarTicket(Vticket).Trim();
    if (!string.IsNullOrEmpty(ticketTexto))
    {
        membrete = membrete + "\n" + ticketTexto + "\n";
    }

    string[] datosmenu = { "1.-Agregar ", "2.-Eliminar ", "3.-Listado ", "4.-Pagar ", "5.-Salida " };
    opcion = DesplegarMenu(membrete + "\n Menu de Punto de Venta", datosmenu);

    if (opcion == null)
    {
        Console.WriteLine("dato incorrecto introducido");
    }
    else
    {
        switch (opcion)
        {
            case "1":
```

```
CapturaVentaProducto(Vticket, productos, idticket, tamticket);
```

```
break;
```

```
case "2":
```

```
    Eliminar(Vticket, productos);
```

```
break;
```

```
case "3":
```

```
    if (ObtenerUltimaPosicion(Vticket) > -1)
```

```
    {
```

```
        Console.WriteLine(MostrarTicket(Vticket));
```

```
    }
```

```
break;
```

```
case "4":
```

```
    Console.WriteLine(MostrarTicketVenta(Vticket, idticket, fechadia, productos).Trim());
```

```
    Pagar(idticket, ventas, Vticket);
```

```
    pago = true;
```

```
    opcion = "5";
```

```
break;
```

```
case "5":
```

```
    Console.WriteLine("Salida del Ventas");
```

```
    if (!pago)
```

```
    {
```

```
        Console.WriteLine("No pago el ticket");
```

```
        CancelarVenta(Vticket, productos);

        Console.WriteLine("eliminando tickte " + idticket);
    }

    break;

default:

    Console.WriteLine("No existe esta opcion");

    break;
}

}

}

while (opcion.CompareTo("5") != 0);
}

public static string MostrarVenta(string[] venta)
{
    string idticket = RellenarEspacios(venta[0], 6);
    string codigo = RellenarEspacios(venta[1], 5);
    string producto = RellenarEspacios(venta[2], 30);
    string precio = RellenarEspacios(venta[3], 10);
    string cantidad = RellenarEspacios(venta[4], 10);

    string cadena = idticket + codigo + producto + precio + cantidad;

    return cadena;
}
```

```
public static string MostrarListaVentas(string[][] ventas)
```

```
{
```

```
    int posventas = ObtenerUltimaPosicion(ventas);
```

```
    string salida = "";
```

```
    for (int ciclo = 0; ciclo <= posventas; ciclo++)
```

```
    {
```

```
        string[] venta =
```

```
        {
```

```
            ventas[ciclo][0],
```

```
            ventas[ciclo][1],
```

```
            ventas[ciclo][2],
```

```
            ventas[ciclo][3],
```

```
            ventas[ciclo][4]
```

```
        };
```

```
        string cadena = MostrarVenta(venta);
```

```
        salida = salida + cadena + "\n";
```

```
    }
```

```
    return salida;
```

```
}
```

```
public static void AgregarStock(string[][] vproductos)
```

```
{
```

```
    string codigo, cantidad;
```

```
int posicion;
```

```
string info = MostrarLista(vproductos);
```

```
codigo = LeerValidado(info + "\nIntroduce el codigo del producto a modificar", 1);
```

```
if (codigo != null)
```

```
{
```

```
    posicion = ExisteProducto(codigo, vproductos);
```

```
    if (posicion > -1)
```

```
    {
```

```
        string[] vproducto =
```

```
        {
```

```
            vproductos[posicion][0],
```

```
            vproductos[posicion][1],
```

```
            vproductos[posicion][2],
```

```
            vproductos[posicion][3],
```

```
            vproductos[posicion][4]
```

```
        };
```

```
cantidad = LeerValidado("\nIntroduce la cantidad de stock a agregar ", 1);
```

```
int stockActual = int.Parse(vproductos[posicion][3]);
```

```
int stockNuevo = int.Parse(cantidad);
```

```
vproductos[posicion][3] = (stockActual + stockNuevo).ToString();
```

```
        Console.WriteLine("stock actualizado");
    }
    else
    {
        Console.WriteLine("no existe el codigo");
    }
}
else
{
    Console.WriteLine("dato nulo");
}
}

public static void MenuInventario(string[][] vproductos)
{
    string[] datosmenuinventario = { "1.-Listado ", "2.-Agregar ", "3.-Salida " };
    string opcion = "0";

    do
    {
        opcion = DesplegarMenu("Opciones de Inventarios", datosmenuinventario);

        if (opcion == null)
        {
            Console.WriteLine("opcion incorrecta ");
        }
    }
}
```

```
}  
else  
{  
    switch (opcion)  
    {  
        case "1":  
            Console.WriteLine(MostrarLista(vproductos));  
            break;  
  
        case "2":  
            AgregarStock(vproductos);  
            break;  
  
        case "3":  
            Console.WriteLine("Salida del Sistema ");  
            break;  
  
        default:  
            Console.WriteLine("No existe esta opcion ");  
            break;  
    }  
}  
}  
while (opcion.CompareTo("3") != 0);  
}
```

```
public static void MenuPrincipal(string[][] vproductos, string[][] vventas)
{
    string[] datosmenuprincipal = { "1.-Productos ", "2.-Punto de Venta ", "3.- Inventario", "4.-Ventas", "5.-Salida " };
    string opcion = "0";
    string idticket;

    do
    {
        idticket = ObtenerUltimoValorVentas(vventas);

        opcion = DesplegarMenu("Menu de Punto de Venta Cafeteria Coffee House", datosmenuprincipal);

        if (opcion == null)
        {
            Console.WriteLine("opcion incorrecta ");
        }
        else
        {
            switch (opcion)
            {
                case "1":
                    MenuProductos(vproductos);
                    break;

                case "2":
                    MenuPuntoVenta(vventas, idticket, vproductos);
```

```
break;
```

```
case "3":
```

```
    MenuInventario(vproductos);
```

```
    break;
```

```
case "4":
```

```
    Console.WriteLine(MostrarListaVentas(vventas));
```

```
    break;
```

```
case "5":
```

```
    Console.WriteLine("Salida del Sistema ");
```

```
    break;
```

```
default:
```

```
    Console.WriteLine("No existe esta opcion ");
```

```
    break;
```

```
}
```

```
}
```

```
}
```

```
while (opcion.CompareTo("5") != 0);
```

```
}
```

```
public static string ObtenerUltimoValorVentas(string[][] ventas)
```

```
{
```

```
    int ultimaposicion = ObtenerUltimaPosicion(ventas);
```

```
string ultimoValor = "000";
```

```
if (ultimaposicion >= 0)
```

```
{
```

```
    ultimoValor = ventas[ultimaposicion][0];
```

```
}
```

```
return ultimoValor;
```

```
}
```

```
public static void Main(string[] args)
```

```
{
```

```
    productos = CargarProductos();
```

```
    ventas = CrearVenta();
```

```
    MenuPrincipal(productos, ventas);
```

```
}
```

```
}
```